

CSE 4403: Algorithms

Lecture15

Dynamic Programming – III

Part-2

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

0/1 Knapsack

Problem: There are n items, each with **weight[i]** and **value[i]**. Knapsack capacity **W**.

You have to **maximize the value without exceeding capacity**. Each item should **be used at most once**.



0/1 Knapsack

Problem: There are n items, each with **weight[i]** and **value[i]**. Knapsack capacity **W**.

You have to **maximize the value without exceeding capacity**. Each item should **be used at most once**.

State: $dp[i][w]$ = max value using first i items with capacity w



0/1 Knapsack

Recurrence relation:

```
if weight[i] > w:  
    dp[i][w] = dp[i-1][w]        // can't fit, skip  
else:  
    dp[i][w] = max(dp[i-1][w],    // skip item i  
                   dp[i-1][w - weight[i]] + value[i]) // take item i
```

0/1 Knapsack

```
// Base Cases
for (int i = 0; i <= n; i++)
    dp[i][0] = 0;

for (int w = 0; w <= W; w++)
    dp[0][w] = 0;

// Main DP Loop
for (int i = 1; i <= n; i++) {
    for (int w = 1; w <= W; w++) {

        if (weight[i] > w)
            dp[i][w] = dp[i - 1][w];
        else
            dp[i][w] = max(dp[i - 1][w],
                           dp[i - 1][w - weight[i]] + value[i]);
    }
}

cout << dp[n][W] << endl;
```

0/1 Knapsack

```
int weight[] = {1, 3, 4};
```

```
int value[] = {2, 4, 5};
```

```
int n = 3;
```

```
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
1(w=1)	0	2	2	2	2	2
2(w=3)	0	2	2	4	6	6
3(w=4)	0	2	2	4	5	7

0/1 Knapsack: 1D Backward Hack

But, the question is could we skip 2D DP here?

Yes by running the weight loop backward so that the previous row values are there when updating.

```
vector<int> dp(W + 1, 0);

for (int i = 1; i <= n; i++) {
    for (int w = W; w >= weight[i]; w--) {

        dp[w] = max(dp[w],
                    dp[w - weight[i]] + value[i]);
    }
}

cout << dp[W];
```

0/1 Knapsack: Reconstruction

When is 2D DP still useful?

When we want to know which items are picked for weight W knapsack.

0/1 Knapsack: Reconstruction

The Core Idea

We filled the table forward. Now we trace back from $dp[n][W]$.

At each cell $dp[i][w]$, ask:
"Did I take item i or not?"

- If $dp[i][w] == dp[i-1][w]$ (item i was **skipped**)
- Otherwise (item i was **taken**, move to $dp[i-1][w - \text{weight}[i]]$)

Repeat until $i = 0$.

0/1 Knapsack: Reconstruction

```
int weight[] = {1, 3, 4};  
int value[] = {2, 4, 5};
```

```
int n = 3;  
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
Item 1 (1)	0	2	2	2	2	2
Item 2 (3)	0	2	2	4	6	6
Item 3 (4)	0	2	2	4	5	7

Traceback starting at $dp[3][5] = 7$:

$i=3, w=5$:

$dp[3][5]=7$ vs $dp[2][5]=6 \rightarrow$ **not equal** $\rightarrow$ **take item 3 (w=4)**

0/1 Knapsack: Reconstruction

```
int weight[] = {1, 3, 4};  
int value[] = {2, 4, 5};
```

```
int n = 3;  
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
Item 1 (1)	0	2	2	2	2	2
Item 2 (3)	0	2	2	4	6	6
Item 3 (4)	0	2	2	4	5	7

Traceback starting at $dp[3][5] = 7$:

$i=3, w=5$:

$dp[3][5]=7$ vs $dp[2][5]=6 \rightarrow$ **not equal** $\rightarrow$ **take item 3 ($w=4$)**, move to $dp[2][5-4] = dp[2][1]$

0/1 Knapsack: Reconstruction

```
int weight[] = {1, 3, 4};  
int value[] = {2, 4, 5};
```

```
int n = 3;  
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
Item 1 (1)	0	2	2	2	2	2
Item 2 (3)	0	2	2	4	6	6
Item 3 (4)	0	2	2	4	5	7

i=2, w=1:

dp[2][1]=2 vs dp[1][1]=2 → equal → skip item 2

0/1 Knapsack: Reconstruction

```
int weight[] = {1, 3, 4};  
int value[] = {2, 4, 5};
```

```
int n = 3;  
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
Item 1 (1)	0	2	2	2	2	2
Item 2 (3)	0	2	2	4	6	6
Item 3 (4)	0	2	2	4	5	7

i=2, w=1:

dp[2][1]=2 vs dp[1][1]=2 → equal → skip item 2 → go to item 1

0/1 Knapsack: Reconstruction

```
int weight[] = {1, 3, 4};  
int value[] = {2, 4, 5};
```

```
int n = 3;  
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
Item 1 (1)	0	2	2	2	2	2
Item 2 (3)	0	2	2	4	6	6
Item 3 (4)	0	2	2	4	5	7

i=1, w=1:

dp[1][1]=2 vs dp[0][1]=0 → not equal → take item 1 (w=1, v=2)

0/1 Knapsack: Reconstruction

```
int weight[] = {1, 3, 4};  
int value[] = {2, 4, 5};
```

```
int n = 3;  
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
Item 1 (1)	0	2	2	2	2	2
Item 2 (3)	0	2	2	4	6	6
Item 3 (4)	0	2	2	4	5	7

i=1, w=1:

dp[1][1]=2 vs dp[0][1]=0 → **not equal** → **take item 1 (w=1, v=2)**, move to dp[0][1-1] = dp[0][0]

0/1 Knapsack: Reconstruction

```
int weight[] = {1, 3, 4};  
int value[] = {2, 4, 5};
```

```
int n = 3;  
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
Item 1	0	2	2	2	2	2
Item 2	0	2	2	4	6	6
Item 3	0	2	2	4	5	7

i=0: stop

0/1 Knapsack: Reconstruction code

```
int i = n;
int w = W;

vector<int> chosen;

while (i > 0 && w > 0) {
    if (dp[i][w] == dp[i - 1][w]) {
        i--;          // Item i not taken
    }
    else {
        chosen.push_back(i);    // Item i taken
        w -= weight[i];
        i--;
    }
}
```

LCS

Given: Two strings s1 and s2

Find: The length of their **Longest Common Subsequence**

- A subsequence = characters in order, but not necessarily contiguous.

LCS

Given: Two strings s1 and s2

Find: The length of their **Longest Common Subsequence**

- A subsequence = characters in order, but not necessarily contiguous.

Example:

- s1 = "ABCBDAB"
- s2 = "BDCABA"
- LCS = "BCBA" or "BDAB" → length = 4

Note: LCS ≠ Longest Common Substring (which must be contiguous)

LCS

Given: Two strings s1 and s2

Find: The length of their **Longest Common Subsequence**

- A subsequence = characters in order, but not necessarily contiguous.

How do we solve this?

What would be the Brute force method?

Think of how many subsequences does a string “ABCB DAB” has.

We have to match that many subsequences with the other string.

Time complexity?

LCS: DP Solution

State:

$dp[i][j]$ = length of LCS of first i chars of $s1$ and first j chars of $s2$

Recurrence:

if $s1[i] == s2[j]$:

$$dp[i][j] = dp[i-1][j-1] + 1$$

else:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

Base case:

$dp[0][j] = 0$ and $dp[i][0] = 0$ for all i, j
(LCS with an empty string is always 0)

Answer: $dp[m][n]$ where $m = |s1|$, $n = |s2|$

LCS: DP Solution Intuition

- $s1[i] == s2[j]$:
 - these two characters must be in the LCS. Take them and look at what remains: $dp[i-1][j-1] + 1$
- $s1[i] != s2[j]$:
 - at least one of them is NOT in the LCS.
 - Try skipping each
 - Skip $s1[i] \rightarrow dp[i-1][j]$
 - Skip $s2[j] \rightarrow dp[i][j-1]$
 - Take the better option: max of both

LCS: Top Down

```
int LCS(string X, string Y, int i, int j) {  
  
    // Base Case  
    if (i == 0 || j == 0)  
        return 0;  
  
    // Characters match  
    if (X[i - 1] == Y[j - 1])  
        return 1 + LCS(X, Y, i - 1, j - 1);  
  
    // Characters don't match  
    return max(LCS(X, Y, i - 1, j),  
               LCS(X, Y, i, j - 1));  
}
```

LCS: Top Down DP

```
int LCS(string X, string Y, int i, int j) {  
  
    // Base Case  
    if (i == 0 || j == 0)  
        return 0;  
  
    // Already computed  
    if (dp[i][j] != -1)  
        return dp[i][j];  
  
    // Characters match  
    if (X[i - 1] == Y[j - 1])  
        return dp[i][j] = 1 + LCS(X, Y, i - 1, j - 1);  
  
    // Characters don't match  
    return dp[i][j] = max(LCS(X, Y, i - 1, j),  
                          LCS(X, Y, i, j - 1));  
}
```

LCS: Bottom Up

```
vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

// Base Cases
for (int i = 0; i <= m; i++)
    dp[i][0] = 0;

for (int j = 0; j <= n; j++)
    dp[0][j] = 0;

// Main DP Loop
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {

        if (X[i - 1] == Y[j - 1])
            dp[i][j] = 1 + dp[i - 1][j - 1];
        else
            dp[i][j] = max(dp[i - 1][j],
                           dp[i][j - 1]);
    }
}

cout << dp[m][n];
```

LCS: DP Table

s1 = "ABBCD", s2 = "BABAC"

s2

s1

i \ j		s2					
		""	B	A	B	A	C
""							
A							
B							
B							
C							
D							

LCS: DP Table

s1 = "ABBCD", s2 = "BABAC"

s2

s1

i \ j	""	s2					
		""	B	A	B	A	C
""	0	0	0	0	0	0	0
A	0	0	0	1	1	1	1
B	0	0	1	1	2	2	2
B	0	0	1	1	2	2	2
C	0	0	1	1	2	2	3
D	0	0	1	1	2	2	3

Homework

1. Reconstruction for LCS:
What is the longest common subsequence?
2. Edit Distance

```
string reconstructLCS(string s1, string s2,  
                     vector<vector<int>>& dp) {  
  
    int i = s1.size();  
    int j = s2.size();  
  
    string lcs = "";  
  
    while (i > 0 && j > 0) {  
  
        // Characters match  
        if (s1[i - 1] == s2[j - 1]) {  
            lcs += s1[i - 1];  
            i--;  
            j--;  
        }  
        // Move up  
        else if (dp[i - 1][j] >= dp[i][j - 1]) {  
            i--;  
        }  
        // Move Left  
        else {  
            j--;  
        }  
    }  
  
    reverse(lcs.begin(), lcs.end());  
    return lcs;  
}
```